



Deep Learning

Tutorial 10: Segmentation using U-NET

(Custom Model)

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

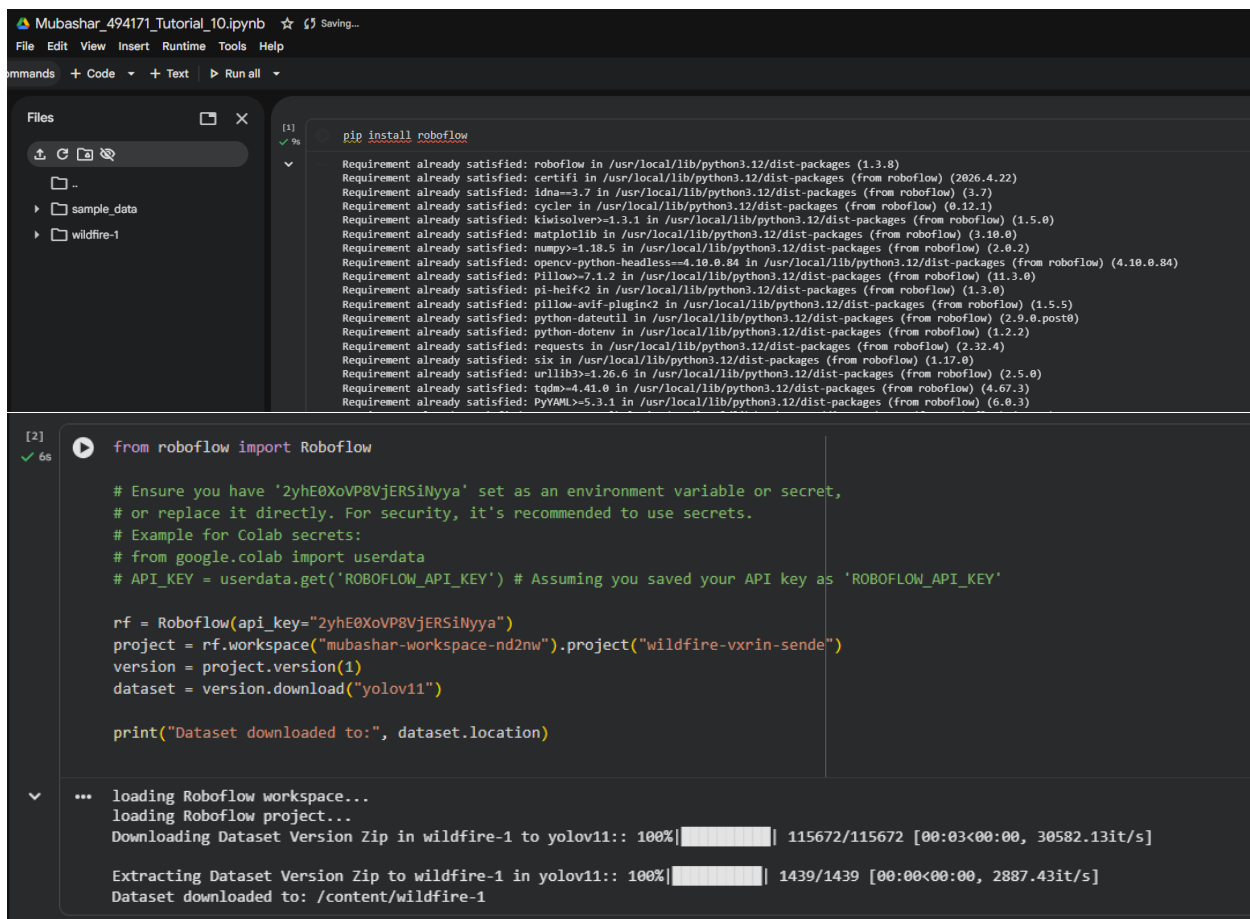
School of Mechanical & Manufacturing Engineering

NUST

Tasks:

- Develop your own data through labelling
- Use the data you developed and test it on the model developed and change the layers.
- Use different learning rates and visualize the results

Implementation of these tasks are in PyTorch:



```
Mubashar_494171_Tutorial_10.ipynb ☆ Saving...
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all ▾

Files
└─ .
└─ sample_data
└─ wildfire-1

[1] pip install roboflow
Requirement already satisfied: roboflow in /usr/local/lib/python3.12/dist-packages (1.3.8)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from roboflow) (2026.4.22)
Requirement already satisfied: idna<3.7 in /usr/local/lib/python3.12/dist-packages (from roboflow) (3.7)
Requirement already satisfied: cyclor in /usr/local/lib/python3.12/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver<1.3.1 in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.5.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from roboflow) (3.10.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.0.2)
Requirement already satisfied: opencv-python-headless<=4.10.0.84 in /usr/local/lib/python3.12/dist-packages (from roboflow) (4.10.0.84)
Requirement already satisfied: Pillow<7.1.2 in /usr/local/lib/python3.12/dist-packages (from roboflow) (11.3.0)
Requirement already satisfied: pi-helfc2 in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.3.0)
Requirement already satisfied: pillow-avif-plugin2 in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.5.5)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.9.0.post0)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.2.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.32.4)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.17.0)
Requirement already satisfied: urllib3<=1.26.6 in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.5.0)
Requirement already satisfied: tqdm<=4.41.0 in /usr/local/lib/python3.12/dist-packages (from roboflow) (4.67.3)
Requirement already satisfied: PyYAML<=5.3.1 in /usr/local/lib/python3.12/dist-packages (from roboflow) (6.0.3)

[2] from roboflow import Roboflow

# Ensure you have '2yhE0XoVP8VjERSiNyYa' set as an environment variable or secret,
# or replace it directly. For security, it's recommended to use secrets.
# Example for Colab secrets:
# from google.colab import userdata
# API_KEY = userdata.get('ROBOFLOW_API_KEY') # Assuming you saved your API key as 'ROBOFLOW_API_KEY'

rf = Roboflow(api_key="2yhE0XoVP8VjERSiNyYa")
project = rf.workspace("mubashar-workspace-nd2nw").project("wildfire-vxrin-sende")
version = project.version(1)
dataset = version.download("yolov11")

print("Dataset downloaded to:", dataset.location)

... loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in wildfire-1 to yolov11:: 100%|██████████| 115672/115672 [00:03<00:00, 30582.13it/s]

Extracting Dataset Version Zip to wildfire-1 in yolov11:: 100%|██████████| 1439/1439 [00:00<00:00, 2887.43it/s]
Dataset downloaded to: /content/wildfire-1
```

1. Environment Setup and Imports

First, we'll import all the necessary libraries and set up the device (CPU or GPU if available).

```
[3]
✓ 10s
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
import glob

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

... Using device: cuda

2. Custom Dataset for YOLOv11 Segmentation

Roboflow's YOLOv11 export provides images and corresponding label files (`.txt`) containing polygon coordinates for segmentation masks. We need a custom `Dataset` to parse these text files and create binary masks. We'll also apply random transformations as requested.

Assumed directory structure after Roboflow download:

```
/content/wildfire-1/
├─ train/
│  ├─ images/
│  └─ labels/
├─ valid/
│  ├─ images/
│  └─ labels/
└─ test/
   ├─ images/
   └─ labels/
```

```

[5]
✓ Os  import cv2 # Import cv2 for polygon filling

# Define a global image size for consistency
img_size = (256, 256) # Define img_size here

class WildfireSegmentationDataset(Dataset):
    def __init__(self, img_dir, label_dir, img_transform=None, mask_transform=None, img_size=img_size):
        self.img_dir = img_dir
        self.label_dir = label_dir
        self.img_transform = img_transform
        self.mask_transform = mask_transform
        self.img_size = img_size
        self.image_files = sorted(glob.glob(os.path.join(img_dir, "*.jpg")) + \
                                   glob.glob(os.path.join(img_dir, "*.png")))

    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
        img_path = self.image_files[idx]
        img = Image.open(img_path).convert("RGB")
        original_width, original_height = img.size

        # Create an empty mask
        mask = np.zeros((original_height, original_width), dtype=np.uint8)

        label_filename = os.path.basename(img_path).replace(".jpg", ".txt").replace(".png", ".txt")
        label_path = os.path.join(self.label_dir, label_filename)

        if os.path.exists(label_path):
            with open(label_path, 'r') as f:
                for line in f:
                    parts = list(map(float, line.strip().split()))
                    class_id = int(parts[0])
                    # For binary segmentation, we treat all segmented classes as one (wildfire)
                    if class_id == 0: # Assuming class 0 is 'wildfire' or the target class
                        # YOLO format: class_id, x1, y1, x2, y2, ..., xn, yn (normalized)
                        # Convert normalized polygon points to absolute coordinates
                        polygon_points = []
                        for i in range(1, len(parts), 2):
                            x = int(parts[i] * original_width)
                            y = int(parts[i+1] * original_height)
                            polygon_points.append((x, y))

                        # Fill the polygon on the mask
                        points = np.array(polygon_points, dtype=np.int32)
                        cv2.fillPoly(mask, [points], 255) # Fill with white (255)

        mask = Image.fromarray(mask) # Convert numpy array mask to PIL Image

        # Apply transformations (random augmentations)
        seed = np.random.randint(2147483647) # Make a random seed for consistent transforms

        if self.img_transform:
            torch.manual_seed(seed) # Apply same seed for img and mask for consistent geometric transforms
            img = self.img_transform(img)

        if self.mask_transform:
            torch.manual_seed(seed) # Apply same seed for img and mask for consistent geometric transforms
            mask = self.mask_transform(mask)

```

```

        # Ensure mask is binary and single channel float tensor
        mask = (mask > 0).float() # Binary mask (0.0 or 1.0)
        if mask.ndim == 2: # If ToTensor() returned [H, W], make it [1, H, W]
            mask = mask.unsqueeze(0)

        return img, mask

    # Define transformations for images (with normalization and data augmentation)
    train_img_transform = transforms.Compose([
        transforms.Resize(img_size), # Resize images
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomRotation(degrees=15),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
        transforms.ToTensor(), # Converts to tensor and scales pixels to [0, 1]
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # ImageNet normalization
    ])

    val_img_transform = transforms.Compose([
        transforms.Resize(img_size),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    # Define transformations for masks (without normalization, just resize and ToTensor)
    mask_transform = transforms.Compose([
        transforms.Resize(img_size), # Resize masks
        transforms.ToTensor(), # Converts to tensor, scales pixels to [0, 1]
    ])

```

```

[5]
✓ Os
# Dataset directories (adjust if your Roboflow export is structured differently)
data_root = "/content/wildfire-1"
train_img_dir = os.path.join(data_root, "train", "images")
train_label_dir = os.path.join(data_root, "train", "labels")
val_img_dir = os.path.join(data_root, "valid", "images")
val_label_dir = os.path.join(data_root, "valid", "labels")

# Create datasets, passing separate image and mask transforms
train_dataset = WildfireSegmentationDataset(train_img_dir, train_label_dir, img_transform=train_img_transform, mask_transform=mask_transform)
val_dataset = WildfireSegmentationDataset(val_img_dir, val_label_dir, img_transform=val_img_transform, mask_transform=mask_transform)

# Create DataLoaders
batch_size = 8 # You can adjust this based on your GPU memory
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=2)

print(f"Training images: {len(train_dataset)}")
print(f"Validation images: {len(val_dataset)}")

# Test loading a sample to ensure it works
if len(train_dataset) > 0:
    sample_img, sample_mask = train_dataset[0]
    print(f"Sample Image shape: {sample_img.shape}")
    print(f"Sample Mask shape: {sample_mask.shape}")
    print(f"Sample Mask unique values: {np.unique(sample_mask.numpy())}")

# Visualize a sample
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
# Define mean and std for denormalization inside the visualization block
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]
plt.imshow(sample_img.permute(1, 2, 0).cpu().numpy() * np.array(std) + np.array(mean)) # Denormalize for display
plt.title("Sample Image")
plt.axis('off')

```

```

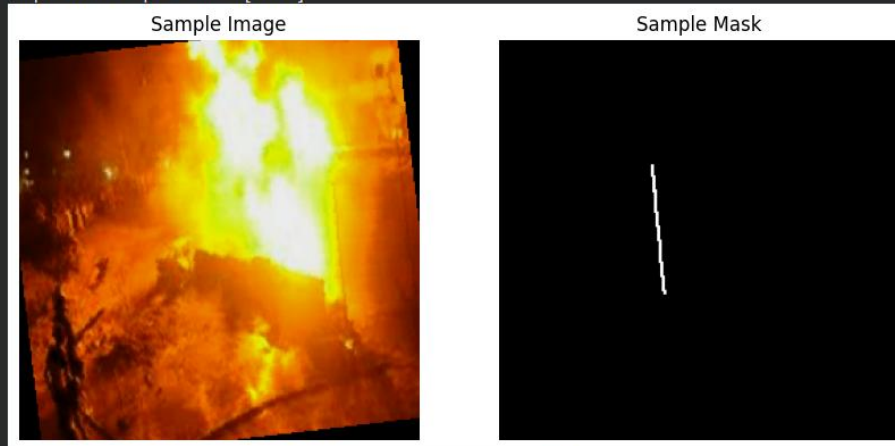
plt.subplot(1, 2, 2)
plt.imshow(sample_mask.squeeze().cpu().numpy(), cmap='gray')
plt.title("Sample Mask")
plt.axis('off')
plt.show()
else:
    print("No training samples found to display.")

```

```

*** WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers)
Training images: 502
Validation images: 143
Sample Image shape: torch.Size([3, 256, 256])
Sample Mask shape: torch.Size([1, 256, 256])
Sample Mask unique values: [0. 1.]

```



3. Define the U-Net Model

Here, we'll implement a basic U-Net architecture, which is a popular choice for image segmentation tasks. This U-Net will have a contracting path (encoder) and an expansive path (decoder) with skip connections.

```

[6] class DoubleConv(nn.Module):
    """(convolution => BN => ReLU) * 2"""
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.double_conv(x)

class Down(nn.Module):
    """Downscaling with maxpool then double conv"""
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.maxpool_conv = nn.Sequential(
            nn.MaxPool2d(2),
            DoubleConv(in_channels, out_channels)
        )

    def forward(self, x):
        return self.maxpool_conv(x)

```

```

class Up(nn.Module):
    """Upscaling then double conv"""
    def __init__(self, in_channels, out_channels, bilinear=True):
        super().__init__()

        # if bilinear, use the normal convolutions to reduce the number of channels
        if bilinear:
            self.up = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
            self.conv = DoubleConv(in_channels, out_channels) # No reduction in channels here, DoubleConv handles it
        else:
            self.up = nn.ConvTranspose2d(in_channels, in_channels // 2, kernel_size=2, stride=2)
            self.conv = DoubleConv(in_channels, out_channels)

    def forward(self, x1, x2):
        x1 = self.up(x1)
        # input is CHW
        diffY = x2.size()[2] - x1.size()[2]
        diffX = x2.size()[3] - x1.size()[3]

        x1 = nn.functional.pad(x1, [diffX // 2, diffX - diffX // 2,
                                     diffY // 2, diffY - diffY // 2])
        x = torch.cat([x2, x1], dim=1)
        return self.conv(x)

class OutConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super(OutConv, self).__init__()
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=1)

    def forward(self, x):
        return self.conv(x)

```

```

class UNet(nn.Module):
    def __init__(self, n_channels, n_classes, bilinear=False):
        super(UNet, self).__init__()
        self.n_channels = n_channels
        self.n_classes = n_classes
        self.bilinear = bilinear

        self.inc = DoubleConv(n_channels, 64)
        self.down1 = Down(64, 128)
        self.down2 = Down(128, 256)
        self.down3 = Down(256, 512)
        factor = 2 if bilinear else 1
        self.down4 = Down(512, 1024 // factor)
        self.up1 = Up(1024, 512 // factor, bilinear)
        self.up2 = Up(512, 256 // factor, bilinear)
        self.up3 = Up(256, 128 // factor, bilinear)
        self.up4 = Up(128, 64, bilinear)
        self.outc = OutConv(64, n_classes)

    def forward(self, x):
        x1 = self.inc(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.down3(x3)
        x5 = self.down4(x4)
        x = self.up1(x5, x4)
        x = self.up2(x, x3)
        x = self.up3(x, x2)
        x = self.up4(x, x1)
        logits = self.outc(x)
        return logits

```

```
# Initialize the U-Net model
model = UNet(n_channels=3, n_classes=1) # 3 input channels (RGB), 1 output channel (binary mask)
model.to(device)

print("U-Net model initialized:")
print(model)
```

```
)
)
(up1): Up(
  (up): ConvTranspose2d(1024, 512, kernel_size=(2, 2), stride=(2, 2))
  (conv): DoubleConv(
    (double_conv): Sequential(
      (0): Conv2d(1024, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU(inplace=True)
      (3): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (4): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (5): ReLU(inplace=True)
    )
  )
)
```

4. Training and Validation Loop

Now we'll define the loss function, optimizer, and implement the training and validation loops for 5 epochs using the specified learning rate of 0.001.

```
[7] 2m
# Loss function and optimizer
# BCEWithLogitsLoss is good for binary segmentation as it combines Sigmoid and BCELoss
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

num_epochs = 5

train_losses = []
val_losses = []

for epoch in range(num_epochs):
    model.train() # Set model to training mode
    running_loss = 0.0
    for batch_idx, (inputs, masks) in enumerate(train_loader):
        inputs = inputs.to(device)
        masks = masks.to(device) # Masks should be on the same device as inputs

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, masks)
        loss.backward()
        optimizer.step()
```




5. Inference and Visualization

Finally, let's take a look at how our trained model performs on a few images from the validation set. We'll visualize the original image, the ground truth mask, and the predicted mask.

```
[8]
✓ 1s model.eval() # Set model to evaluation mode

def visualize_prediction(model, dataset, device, num_samples=3):
    plt.figure(figsize=(15, 5 * num_samples))
    for i in range(num_samples):
        # Get a random sample from the validation dataset
        idx = np.random.randint(0, len(dataset))
        image, true_mask = dataset[idx]

        # Add batch dimension and move to device
        input_image = image.unsqueeze(0).to(device)

        # Make prediction
        with torch.no_grad():
            output = model(input_image)

        # Convert output to binary mask (sigmoid and threshold)
        predicted_mask = torch.sigmoid(output).squeeze().cpu().numpy()
        predicted_mask = (predicted_mask > 0.5).astype(np.uint8)

        # Denormalize image for display
        image_display = image.permute(1, 2, 0).cpu().numpy()
        # Assuming ImageNet normalization, reverse it
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        image_display = std * image_display + mean
        image_display = np.clip(image_display, 0, 1)

        # Plotting
        plt.subplot(num_samples, 3, i * 3 + 1)
        plt.imshow(image_display)
        plt.title("Original Image")
        plt.axis('off')

        plt.subplot(num_samples, 3, i * 3 + 2)
        plt.imshow(true_mask.squeeze().cpu().numpy(), cmap='gray')
        plt.title("Ground Truth Mask")
        plt.axis('off')

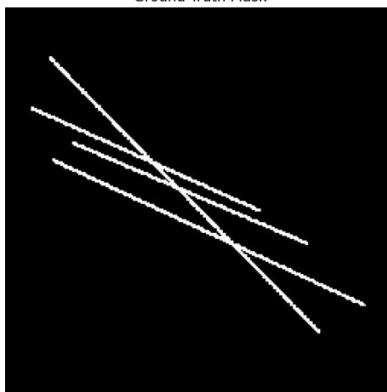
        plt.subplot(num_samples, 3, i * 3 + 3)
        plt.imshow(predicted_mask, cmap='gray')
        plt.title("Predicted Mask")
        plt.axis('off')
    plt.tight_layout()
    plt.show()

visualize_prediction(model, val_dataset, device, num_samples=3)
```

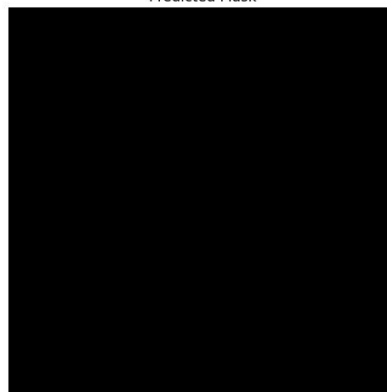
Original Image



Ground Truth Mask



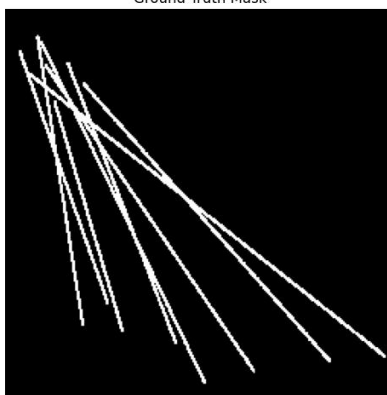
Predicted Mask



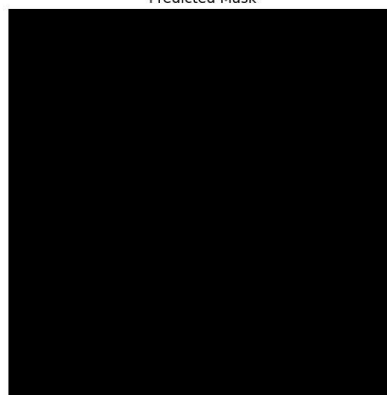
Original Image



Ground Truth Mask



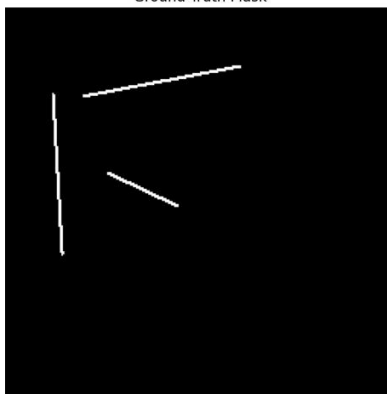
Predicted Mask



Original Image



Ground Truth Mask



Predicted Mask

